



Retriever

Open source autonomous outdoor mobile robot

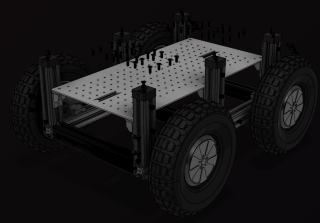
[Sponsorship dossier](#) · [Technical project brief](#)

William Hanczyk
KEDGE Business School
Bordeaux, France

wcontact33@gmail.com
github.com/WilliamH07/retriever

August 2026
Version 1.0

Hardware and software released under
an open source licence.



Project overview

Retriever is a four wheel drive outdoor mobile robot, built independently and released as open hardware. Its single objective is autonomous outdoor navigation using ROS 2 Jazzy and the Nav2 stack, on a chassis that anyone can reproduce from documented, commodity parts.

The project is an open rebuild inspired by the **Clearpath Husky**, a research platform that has become a reference in mobile robotics laboratories. The Husky is an excellent machine and a commercial product priced accordingly. Retriever asks a different question: how much of that capability can be reached by one person, from recovered and off the shelf components, if the engineering is done properly rather than cheaply? The mechanical design was modelled from scratch, and the electrical and software architecture was designed independently.

What makes the project unusual is not the hardware, which is deliberately ordinary. It is the discipline applied to it: a complete architecture dossier written before the first part was cut, a documented failure mode analysis, a hardware safety chain that no software can override, and a build plan with a measurable exit criterion at every stage.



Figure 1: Rear three quarter view of the current assembly

The goal

The goal is a robot that can be given a destination and reach it on its own: outdoors, on ground nobody prepared for it, carrying a load on its deck, with no operator in the loop and no track to follow.

That target is deliberately narrow, because it is the one that forces every hard problem to be solved at once. Outdoors means the sensors have to work in sunlight and the localisation has to survive without walls to map. Unprepared ground means the drivetrain has to have torque in reserve and the navigation has to react to obstacles that appear on no map. No operator means the machine has to decide, on its own, when to stop.

Success is defined by things that can be measured rather than demonstrated:

Navigation	Reach a GNSS waypoint outdoors, autonomously, and report arrival
Obstacles	Detect and avoid an obstacle that is on no map, while under way
Speed	1.5 m/s sustained
Endurance	70 minutes of continuous operation on one charge
Safety	Come to a stop in under 500 ms on loss of radio link, loss of computer, loss of a controller, or the emergency stop, every time
Openness	Reproducible by someone else from the published files alone

The last line is not decoration. A robot that only works in the hands of the person who built it has not been engineered, it has been fiddled with until it ran. The whole documentation discipline described below exists to make the difference measurable.

Technical specification

Configuration	4WD skid steer, outdoor
Structure	Aluminium extrusion frame, laser cut panels, 3D printed mounts
Target mass	35 kg in running order
Target speed	1.5 m/s
Drivetrain	4 × 6.5 inch hub motors, 250 W each
Motor drivers	4 × BLDC controllers, 36 to 48 V
Battery	36 V, 280 Wh lithium ion, 10S3P
Endurance	70 min at 200 W average
Compute	x86 SBC, Ubuntu 24.04, ROS 2 Jazzy
Real time	3 × ESP32 on CAN 2.0A, 500 kbit/s
Localisation	GNSS, IMU and wheel odometry, dual EKF
Navigation	Nav2, collision monitor, twist mux
Licence	Open hardware and software

System architecture

Three decisions define the platform.

A CAN bus, not USB, carries every command.

Four BLDC controllers chopping tens of amps within 30 cm of the signal wiring make USB the wrong choice for the control path: it is a master slave bus, dynamically enumerated, with no frame priority. CAN 2.0A at 500 kbit/s links the computer to every microcontroller instead. USB is retained only for high bandwidth perception sensors, whose loss degrades the mission but cannot cause dangerous motion.

The main computer never does real time. It runs perception, localisation, Nav2 and the mission state machine, and it produces **setpoints**. A dedicated safety microcontroller decides whether those setpoints may be executed. The computer requests arming; the safety controller grants or refuses it. A ROS 2 bug, an out of memory kill or a blocked kernel cannot produce movement.

Safety is hardware, and independent of software.

Six stop levels, from a 100 ms software collision monitor down to a mechanically latching emergency stop wired in series with the main DC contactor coil. The last two levels contain no code at all. No software, no microcontroller and no logic supply can hold that contactor closed once the mushroom button is pressed.

Where the project stands

Design is complete. The full architecture dossier runs to eight documents covering electrical design, communications, ROS 2 architecture, safety, network, code organisation, test strategy and a costed bill of materials. The mechanical design is finished in CAD.

Construction is well under way. The aluminium extrusion frame is assembled, the printed body panels and wheel arches are fitted, the four hub motors are mounted, the battery pack sits in its compartment and the cooling fans are in place. What remains before the power chain can be energised is the protection and switching hardware, and the wiring.

Stage	Exit criterion	State
1	Architecture and CAD design complete	done
2	Chassis fabricated and assembled	current
3	Power chain energised, no motor connected	next
4	Motors driven, wheels raised, all stop tests pass	planned
5	Autonomous outdoor navigation to a waypoint	planned

Stage 5 is where the hardware described below is needed, and it is the stage this project cannot reach on a student budget alone.

Beyond stage 5, the intention is to mount my Niryo One arm on Retriever and turn the platform into a mobile manipulator: a robot that navigates to a point and then does something once it arrives. The architecture was designed for this from the start. Adding a sixth node to the CAN bus takes two wires and a busbar tap, not a rewire.

Design and build

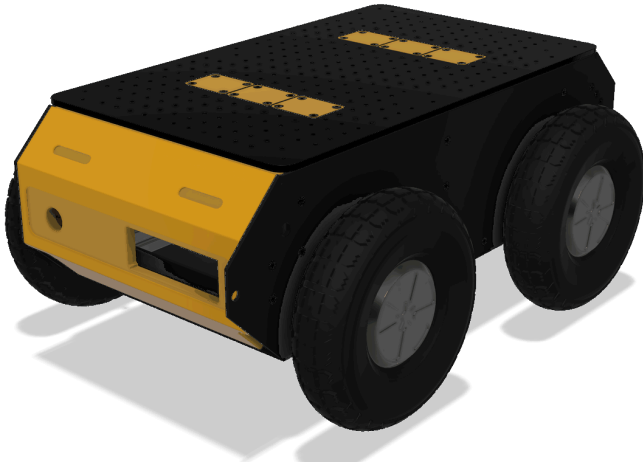


Figure 2: The CAD model of the finished platform



Figure 3: The same machine on the bench, with the battery pack and the four hub motors fitted

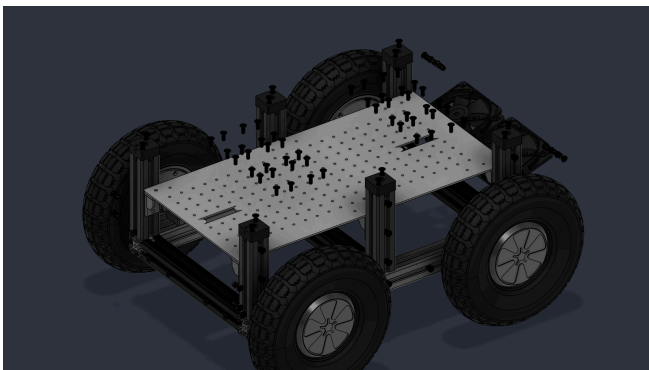


Figure 4: Exploded view with the body panels removed: the two decks, their brackets and the extrusion frame



Figure 5: The assembled base, with the front section and the two cooling fans in place

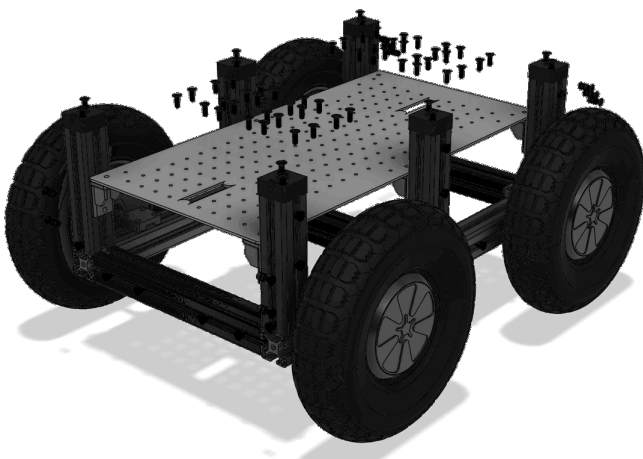


Figure 6: The extrusion frame alone, with the upper deck and its fasteners exploded



Figure 7: Drilling an extrusion for a bracket

Every part of the frame is cut, drilled and printed by hand. The renders on the left of each pair are not illustrations made after the fact: they are the files the parts were made from, and they are published with the rest of the project.

Looking for sponsors

Four things stand between the platform as it is today and a robot that works outdoors on its own. If one of them is something you make, or something you could help with, I would be glad to talk.

Printed circuit boards. Three boards are being designed in EasyEDA: a motor driver interface, a safety and power management board, and a CAN distribution board. They need making.

Embedded AI compute. The current x86 board runs the navigation stack, but not the perception that outdoor autonomy asks for.

An outdoor 3D LiDAR. The blocking item. The 2D LiDAR on hand is specified for indoor use, and direct sunlight saturates its receiver.

A depth camera. The Kinect v2 on hand measures depth by active infrared, so it is blind in daylight, and its driver has been unmaintained since 2021.

Item	What it has to do
PCBs	Three boards, 2 and 4 layer prototypes
Compute	Run perception on board, 8 GB class
3D LiDAR	30 m, 360°, works in direct sunlight
Camera	Stereo depth in daylight, ROS 2 driver

That is the whole gap: somewhere around a thousand euros of hardware between the robot as it stands and a robot that works outdoors on its own.

Hardware is the simplest form of support, but a contribution towards buying one of these items is just as welcome, and I am glad to buy whichever model you would rather see on the robot. I am not asking for a particular product. If you would rather propose something from your own range, a discount, or simply advice on what would suit, that is just as useful to me.

What a sponsor receives

Retriever is a documentation project as much as an engineering one, and the documentation is where a sponsor gets value.

Integration	A written and filmed integration tutorial for the sponsored part, published openly. The kind of content that is hard to produce in house and that answers the questions real users ask.
Field test	A field test video of the robot operating outdoors with the sponsored hardware, including honest performance data.
Attribution	Logo on the chassis, in the repository README and on the project page.
Feedback	Honest feedback on how the part behaves in real use, on the robot rather than on a desk. Shared privately first.

About

I am William Hanczyk, a business student at KEDGE Business School in Bordeaux. I have been building robots since childhood, well before I studied anything else. My previous project was a full rebuild of the Niryo One, an open source six axis robotic arm, which taught me most of what I know about motion control, mechanical tolerances and the gap between a CAD model and a machine that actually works.

Retriever is the logical next step, and a considerably harder one: an outdoor platform carrying its own power, its own safety chain and its own autonomy. I build it outside any academic programme, on my own time and my own budget.

The project is deliberately open. Its purpose is to show that a rigorous, safety first mobile robot can be built and documented outside a laboratory, and to leave behind something the next person can reproduce.

Plan view, drawn and built

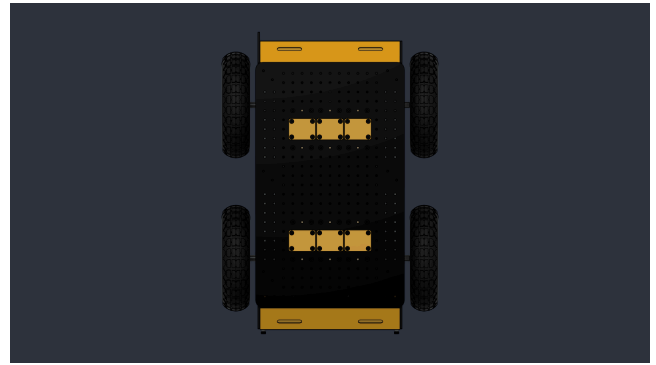


Figure 8: The plan view as drawn



Figure 9: And on the bench: the extrusion frame, the battery pack down the centre with its wiring, and the twin fans at the front

Contact William Hanczyk · wcontact33@gmail.com · github.com/WilliamH07/retriever

Inspired by the Clearpath Husky platform. Mechanical design modelled from scratch; some ROS 2 package conventions follow the open source *husky/husky* repository (BSD-3-Clause). Clearpath Robotics is not affiliated with this project.